

Coding & Solution

Date	15 July 2026
Team ID	TM-RISINGWATERS-04
Project Name	Rising Waters: AI Powered Flood Prediction System
Maximum Marks	5 Marks

Coding & Solution:

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary:

Repository Link / URL	https://github.com/TM-RISINGWATERS-04/rising-waters-core
Programming Language(s)	Python, JavaScript, SQL, HTML/CSS
Framework(s) Used	FastAPI, React, PyTorch, TailwindCSS
Key Features Implemented	• Real-time sensor telemetry processing pipelines (IoT data streams) • Trained UNet deep learning model predicting water surface flow inundations from digital elevation rasters • Interactive Leaflet.js map layer displaying safe versus critical zone routing • Twilio automatic SMS broadcast warning triggers on critical water levels
Pending / Incomplete Features	• Integration with Sentinel-2 live satellite stream API for automated raw geotiff downloads (currently simulated using static pre-downloaded AWS S3 raster assets) • Dynamic offline mode support for native mobile view layout
Setup / Run Instructions	Backend Setup: cd backend && pip install -r requirements.txt && uvicorn main:app --reload Frontend Setup: cd frontend && npm install && npm run dev Database: Run PostgreSQL setup scripts located in /backend/db/init.sql to spin up tables.

Code Quality Checklist:

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The system has been fully optimized using database indexing over spatial geospatial coordinates to speed up query execution time to under 120ms. Future iterations will migrate the prediction pipeline to async background workers using Celery to completely decouple the API web-server thread loops from long-running predictive raster processing tasks.